FOLLOW US

Search

SUBSCRIBE VIA EMAIL

Email Address

Subscribe

Adobe
Acrobat Pro DC
Acrobat's got it.
Try free

Yearly summary

FACEBOOK

OCFreaks.com
1.5 see refined

Be other ad

LPC1768 ADC Programming Tutorial

Posted By Umang Gajera Posted date: October 14, 2017 In Embedded No Comments

Hi folks, in this tutorial we will go through ARM Cortex-M3 LPC1768 ADC programming tutorial. Basically we convert an Analog signal to its Digital version. We will also see LPC17xx ADC Interfacing Example. This also applies to LPC1769 Microcontroller.

Table of Contents

ADC in ARM LPC176x
LPC176x ADC Registers
LPC1768 ADC Modes & Programming
LPC1768/LPC1769 ADC Example

ADC in ARM LPC176x

Analog to Digital Conversion is used when we want to interface an external analog signal or when interfacing analog sensors, like for example a temperature sensor. The ADC block in ARM Cortex-M3 LPC1768 Microcontroller is based on Successive Approximation(SAR) conversion method. LPC1768 ADC Module uses 12-bit, 12 channel, Successive Approximation Analog to Digital converter. It ranges from V_{SSA} to V_{DDA} or commonly from 0V to +3V. Maximum of 8 multiplexed input can be used for ADC.

Pins relating to ADC Module of LPC1768/LPC1769 :

Pin	Description
AD0.0 to AD0.7 (P0.23/24/25/26, P1.30/31, P0.3/2)	Analog input pins. Note from Datasheet: "If ADC is used, signal levels on analog input pins must not be above the level of V_{DDA} at any time. Otherwise, A/D converter readings will be invalid. If the A/D converter is not used in an application then the pins associated with A/D inputs can be used as 5V tolerant digital I/O pins."
V_{REFN}, V_{REFP}	These reference voltage pins used for ADC and DAC.
V_{DDA}, V_{SSA}	V_{DDA} is Analog Power pin and V_{SSA} is Ground pin used to power the ADC module.

Analog Power Pin i.e V_{DDA} must be properly isolated/decoupled from V_{DD}, at bare minimum using a ferrite bead and a decoupling capacitor, to suppress noise present on V_{DD} rail and also to suppress MCU's switching noise which can be induced on the V_{DD} rail.

LPC176x ADC Registers

Lets see registers which are used for ADC programming.

1) **ADCR** – A/D Control Register : This is the main control register for ADC.

1. **BIT(7:0) – SEL:** Bit 7(LSB) this group) is used to select an AD channel of ADC.
2. **BIT(15:8) – CLKDIV:** ADC Peripheral clock i.e. PCLK_ADC0 is divided by (CLKDIV+1) to get the ADC clock. Note that ADC clock speed must be <= 13MHz! As per datasheet specified, must program the smallest value in this field which yields a clock speed of 4.5 MHz or a bit less.
3. **BIT(16) – BURST:** Set to 1 for doing repeated conversions, else 0 for software controlled conversions. Note: START bits must be set to 000 when BURST=1 or conversions will not start. Refer user manual for detailed info.
4. **BIT(21) – PDN :** Set it to 1 for powering up the ADC and making it operational. Set it to 0 for bringing it in powerdown mode.
5. **BIN(26:24) – START:** These bits are used to control the start of ADC conversion when BURST (bit 16) is set to 0. 000 = No start, 001 = Start the conversion now, for other options refer LPC17xx User Manual.
6. **BIT(27) – EDGE:** Set this bit to 1 to start the conversion on falling edge of the selected CAP/MAT signal and set this bit to 0 to start the conversion on rising edge of the selected signal. (Note: This bit is of used only in the case when the START contains a value between 010 to 111 as shown above.)
7. Other bits are reserved.

2) **ADGDR** – A/D Global Data Register : Contains the ADC's flags and the result of the most recent A/D conversion.

1. **BIN(15:4) – RESULT:** When DONE bit = 1, these bits give a binary fraction which represents the voltage on the pin AD0.x, in range V_{SSA} & V_{DDA}. Value of 0x0 indicates that voltage on the given pin was less than, equal to or greater than V_{SSA}, And a value of 0xFFF means that the input voltage was close to, equal to or greater than V_{DDA}.
2. **BIN(29:24) – CHN:** Represents the channel from which RESULT bits were converted. 000= channel 0, 001= channel 1 and so on.
3. **BIN(30) – OVERRUN:** In burst mode this bit is 1 in case of an Overrun i.e. the result of previous conversion being lost/overwritten. This bit will be cleared after reading ADGDR.
4. **BIN(31) – DONE:** When ADC conversion completes this bit is 1. When this register(ADGDR) is read and ADCR is written, this bit gets cleared i.e. set to 0. If ADCR is written while a conversion is in progress then this bit is set and a new conversion is started.
5. Other bits are reserved.

4) **ADDR0 to ADDR7** – A/D Data registers : This register contains the result of the most recent conversion completed on the corresponding channel [0 to 7]. Its structure is same as ADGDR register BIN[31:24] are not used/reserved.

5) **ADSTAT** – A/D Status register : This register contains DONE and OVERRUN flags for all of the A/D channels along with A/D interrupt flag.

1. **BIN(7:0) – DONE7 to 0:** Here 4th bit mirrors DONE status flag from the result register for A/D channel x.
2. **BIN(15:8) – OVERRUN7 to 0:** Even here the 4th bit mirrors OVERRUN status flag from the result register for A/D channel x.
3. **Bit 16 – ADINT:** This bit represents the A/D interrupt flag. It is 1 when any of the individual A/D channel DONE flags is asserted and enabled to contribute to the A/D interrupt via the ADINTEN register below register.
4. Other bits are reserved.

LPC1768 ADC Modes, Setup and Programming

ADC modes in LPC1768 & LPC1769:

1. **Software controlled mode** : In Software mode only one conversion will be done at a time. To perform another conversion you will need to re-initiate the process. In Software mode, only 1 bit in the SEL field of ADCR can be 1 i.e. only 1 Channel(e.g. Pin) can be selected for conversion at a time. Hence conversions can be done only one channel at a time.
2. **Hardware or Burst mode** : In Hardware or Burst mode, conversions are performed continuously on the selected channels in round-robin fashion. Since the conversions cannot be controlled by software, Overrun may occur in this mode. Overrun is the case when a previous conversion result is replaced by new conversion result without previous result being read i.e. the conversion is lost. Usually an interrupt is used in Burst mode to get the latest conversion results. This interrupt is triggered when conversion in one of the selected channel ends.

1A. Setting up and configuring ADC Module for software controlled mode

First, lets define some values which will help us in setting up ADC registers in LPC1768 & LPC1769 ADC block.

```
#define ADC_CLK_EN (1<<12)
#define SEL_AD0_0  (1<<0) /Select Channel AD0.0
#define CLKDIV     1 /# ADC clock divider (ADC_CLOCK=PCLK/(CLKDIV+1)
#define PNDP       (1<<21) //setting it to 0 will power it down
#define START_ZERO (1<<24) //001 for starting the conversion immediately
#define ADC_DONE   (1<<31) //define it as unsigned value or compiler will throw
#define ADCR_SETUP_SCM ((CLKDIV<<8) | PowerUP)
/*SCM = Software Controlled Mode
```

Now we assign ADCR_SETUP to ADCR along with select channel selection information to select channels as required. Finally we assign(by ORing) START_NOW to ADCR to start the conversion process as shown:

```
LPC_SC->PCONP |= ADC_CLK_EN;//enable power to ADC module
LPC->ADC->ADCR = ADCR_SETUP_SCM | SEL_AD0_0;
LPC->ADC->ADCR |= START_CNV;

// OR
LPC_SC->PCONP |= ADC_CLK_EN; //enable power to ADC
LPC->ADC->ADCR = ADCR_SETUP | SEL_AD0_0 | START_CNV;
```

1B. Fetching the conversion result in software controlled mode :

In software controlled mode we continuously monitor bit 31 in the corresponding channel data register ADDR. If bit 31 changes to 1 from 0, it means that current conversion has been completed and the result is ready. For example, if we are using channel 0 of ADC then we monitor for the changes in bit 31 as follows:

```
while(!(LPC_ADC->ADD0 & ADC_DONE) == 0);//this loop will end when bit 31 of AD
```

After this we extract the result which is stored in ADDR bits 4 to 15. Here we right shift ADDR by 4 places and force all other unrelated bits to 0 by using a 12-bit mask value of 0xFFF. 0xFFF is a mask containing 1's at bit locations 0 to 11 and rest 0's. In our case with ADDR0 being used, it can be done as follows :

```
result = (LPC_ADC->ADD0>>4) & 0xFFF;
```

2A. Setting up and configuring ADC Module for Burst mode

Configuring ADC Module is similar to what was done in software controlled mode except here we use the CLKS bits and don't use the START bits in ADCR. ADC_DONE is also not applicable since we are using an ISR which gets triggered when a conversion completes on any of the enabled channels. Additionally, we define the following constants:

```
#define CLKS_0 (0x0) //select Channel AD0.0
#define BURST_ON (1<<16) // 1 for on and 0 for off
#define ADCR_SETUP_BURST ((CLKDIV<<8) | BURST_ON | PNDP)
```

We configure and setup the ADC module in a similar manner(as shown above) as follows :

```
LPC_SC->PCONP |= ADC_CLK_EN;//enable power to ADC
LPC->ADC->ADCR = ADCR_SETUP_BURST | SEL_AD0_0 | SEL_AD0_1;
```

Note that in this case we can select multiple channels for conversion when setting up ADDCR. START bits are not applicable here since the conversions starts as soon we setup ADDCR register.

2B. Fetching the conversion result in Burst mode :

In Burst mode we use an ISR which triggers at the completion of a conversion in any one of the channel. Now, we just need to find the channel for which the conversion was done. For this we fetch the channel number from ADGDR which also stores the conversion result. Bits [26:24] in ADGDR contain the channel number. Hence, we shift it 24 places and use a 3-bit mask value of 0x7 as shown below:

```
unsigned long ADCDR_Read = LPC->ADC->ADGDR;
int channel = (ADCDR_Read>>24) & 0x7;//Extract Channel Number
```

After knowing the Channel number, we have 2 options to fetch the conversion result from. Either we can fetch it from ADGDR or from ADDRx of the corresponding channel. Lets use ADGDR for extracting the conversion result as follows:

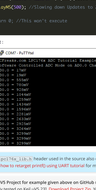
```
int currentResult = (ADCDR_Read>>4) & 0xFFF;//Extract Conversion Result
```

ARM Cortex-M3 LPC1768/LPC1769 ADC Example

Note that in this case no input protection nor filtering was required. But, when interfacing external analog signals it is recommended to use some form of input protection.

Interfacing Potentiometer using ADC on LPC176x

This example performs Analog to Digital conversion in Software Controlled Mode. Here we use P0.23 as analog input for measuring the voltage. P0.23 corresponds to Channel 0 of ADC i.e. AD0.0. For testing I had used a 10K potentiometer for ADC interfacing and connected the middle leg to P0.23 of my LPC1768 development board. You can also use a 5K potentiometer for this ADC Example. Make sure your board V_{REFP}, V_{DDA}, V_{SSA} & V_{REFN} connections. These connections will be generally present on most LPC17xx development boards. If not you can hook it up to V_{CC} and GND respectively using decoupling capacitors and ferrite beads to reduce noise. Schematic for this example is as shown below:



C/C++ Source Code :

```
#include "lpc17xx.h"
#define ADC_CLK_EN (1<<12)
LPC_SC->PCONP |= ADC_CLK_EN;//enable power to ADC module
LPC_ADC->ADCR = ADCR_SETUP_SCM | SEL_AD0_0;
LPC_PINSEL0=(LPC_PINSEL0 | 1<<4|1<<5 ); //Select AD0.0 for P0.23
int ADC_DONE=0;
float volts = 0;

printf("\r\r\rRun LPC17xx ADC Tutorial Example 1.\n\nSoftware Controlled ADC
\n");
while(1)
{
    LPC_ADC->ADCR |= START_CNV;  //Start new Conversion
    while(!(LPC_ADC->ADD0 & ADC_DONE) == 0);//this loop will end when conversion is fini
    result = (LPC_ADC->ADDR0>>4) & 0xFFF; //32 bit Mask to extract result
    volts = (result*VREF)/4096.0;  //Convert result to Voltage
    printf("ADR0.0 = %ew\r\r", (int)(volts*100));//Display with volts
    delay(50);//Slowing down Updates to 2 Updates per second
}
//return 0;//This won't execute
}
```

Serial Output :

ocf_lpc176x_lib.h header used in the source also contains code for retargeting printf(). Check how to merge printf() using LGRT tutorial for more.

KEIL ARM uVS Project for example given above on GitHub @ LPC176x ADC Interfacing Example 1 (Successfully tested on Keil uVS.5). Download Project Zip. You can find the HEX file inside objects folder.

Share this:
Share

Tags: lpc1768

Previous
LPC2148 DAC Programming Tutorial

Next
Western Digital unveils new Innovation in MAMR Technology, 40 TB HDDs by 2025

ABOUT THE AUTHOR

Umang Gajera

ALSO ON OCFREAKS!

The benefits of investing in Metal Fabricators ... 4 months ago
If you are thinking about being a metal fabricator to help you create different ...

OCFreaks! 2 years ago
In this tutorial we see how to create project in KEIL MDK uVision 5 for STM32 ARM ...

OCFreaks! 1 years ago
When thinking of the future, most of us think of flying cars, hoverboards and, ...

OCFreaks! 2 years ago
Who we are W enthusiasts wi for Overclocks

4 Comments

Login

Join the discussion…

LOG IN WITH OR SIGN UP WITH DISQUS ?

Name

Share Best Newest Oldest

Bhim Shankar 8 years ago
Could you please give the code for burst mode also. After getting the channel number by using "int channel = (ADCDR_Read>>24) & 0x7;//Extract Channel Number" how we can get the particular channel ADC value.
Reply

Pavan 8 years ago · edited
Hi admin,
I am sorry I am new in ARM
could you explain to me what means of (+) in LPC_SC->PCONP |= ADC_CLK_EN;
could you give me guide to start learning ARM?
Thanks
Putra
Reply

Power_user_EX MOD → Pavan 5 years ago · edited
Hi Putra, "+" is an operator used to deference pointers. In our case we are deferencing member "PCONP" of pointer to a structure named "LPC_SC".
I have mentioned in http://www.ocfreaks.com/lpc... under GPIO register section.
For more refer : https://en.wikipedia.org/wi...
Regards
Reply

Swan → Power_user_EX 8 years ago
Thank you for your reply, now I am a little understand
I want to know some syntax in LPC17xx.h
I am not detail understand (programming,
could you explain to me what means of this
#define LPC_SC ((LPC_SC_TypeDef *) LPC_SC_BASE)
LPC_SC_BASE is 0x400FC000
this is to defined LPC_SC address, right?
how to translate this code,
do you have a embedded c program language book or document to understand it?
thanks
Reply

Subscribe Privacy Do Not Sell My Data DISQUS

Adobe Keep your creativity going with 20+ apps. Get the Adobe Creative Cloud All Apps plan. Learn more